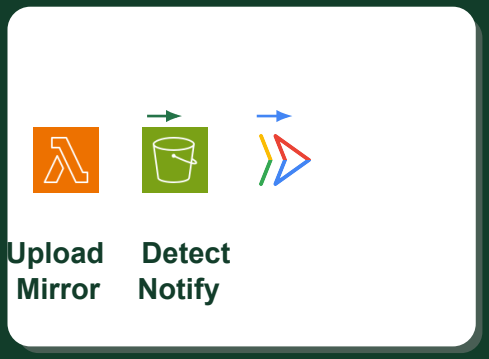
Aussie EcoLens

Multi-Cloud Serverless
Wildlife Observation Platform

AWS Serverless | GCP Model Service | Course ML Tagging



Submission Note

This PDF contains the team report narrative, the multi-cloud architecture diagram, a contribution table template, a user guide, and implementation evidence. Replace the highlighted team-member placeholders with real names, student IDs, and contribution percentages before Moodle submission.

FIT5225 2026 S1 Assignment 2

Team Report Draft for Final Submission

Repository: github.com/ada-yq225/AussieEcoLens

1. Overview

Aussie EcoLens is a multi-cloud online media system for wildlife observation data. Authenticated users can upload camera-trap images and short videos, receive automatic species tags from the supplied course machine-learning model, search media by tags or species, retrieve a full image from a thumbnail URL, manually add or remove tags, delete media, and subscribe to watched-tag notifications. The final deployment uses AWS for authentication, API, storage, database, video frame extraction, and notifications, while Google Cloud hosts the ML inference service and mirrored metadata. This split keeps the application serverless while allowing the large PyTorch, MegaDetector, and classifier runtime to run in a suitable container service.

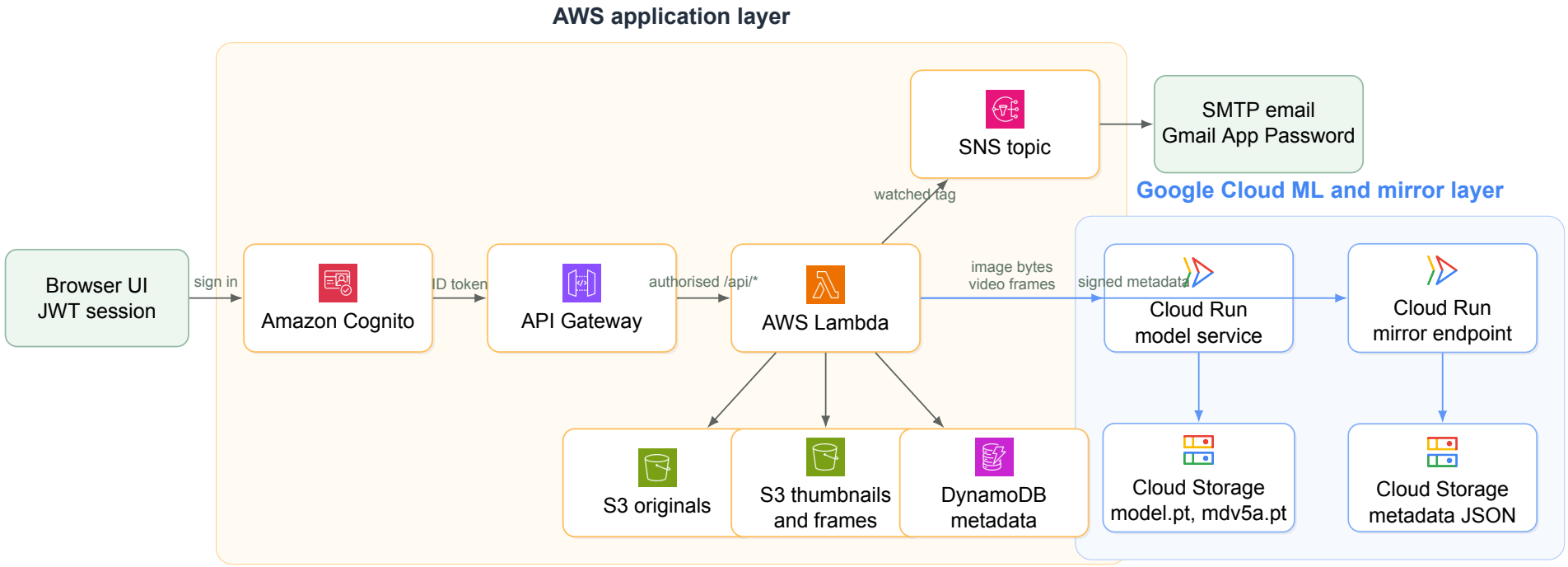
Live Deployment Snapshot

AWS region	`ap-southeast-2`: Cognito, API Gateway, Lambda, S3, DynamoDB, SNS
GCP region	`australia-southeast1`: Cloud Run model service and metadata mirror
ML runtime	Python 3.12, PyTorch, torchvision, MegaDetector, onnx2torch, supplied `model.pt`
Notification	In-app records, SNS publish path, and verified SMTP email through Gmail App Password

Verified Evidence

Cloud smoke testing confirmed unauthenticated API rejection, image upload with `casuarius_casuarius: 1`, duplicate detection, thumbnail-to-full-image lookup, query-by-file without storage, manual tag edit, deletion, video frame extraction at one frame per second, GCP metadata mirroring, and notification channels `in_app`, `sns`, and `smtp`.

2. Multi-Cloud Architecture



The diagram uses official AWS Architecture Icons and official Google Cloud product icons. AWS contains the security and application plane: Cognito protects all workflows, API Gateway verifies JWTs, Lambda processes uploads and queries, S3 stores originals and derivatives, DynamoDB stores searchable metadata, and SNS participates in watched-tag notifications. Google Cloud provides the secondary-cloud plane: Cloud Run loads the supplied model files from Cloud Storage, predicts species for images and video frames, and another Cloud Run endpoint mirrors metadata into Cloud Storage.

3. Design and Implementation Choices

The system is serverless because the workload is event-driven: uploads, queries, tag edits, deletes, and notification watches happen intermittently and should scale without managing virtual machines. Deduplication uses a SHA-256 checksum, so repeated uploads return existing metadata instead of wasting storage. Images receive compressed thumbnails with preserved aspect ratio. Videos are sampled at one frame per second with ffmpeg; each extracted frame is submitted to the same model service and the predictions are aggregated as tag counts. Model handling is flexible: the GCP service reads classifier and detector blob paths from environment variables, so a future model version can be swapped by changing Cloud Storage object names rather than changing Lambda or Cloud Run source code.

4. Core Requirement Coverage

Implemented and Verified

- Authentication and authorisation: AWS Cognito Hosted UI plus API Gateway JWT authoriser; unauthenticated API calls return 401.
- File handling: S3 originals, S3 thumbnails and video frames, checksum deduplication, DynamoDB metadata.
- ML tagging: GCP Cloud Run runs MegaDetector `mdv5a.pt` and the supplied classifier `model.pt`.
- Queries: tag-count AND query, species query, thumbnail-to-full-image lookup, query-by-file without permanent storage.
- Mutations: bulk manual tag add/remove and deletion of originals, thumbnails, frames, and metadata.
- Notifications: watched-tag events create in-app records and publish through SNS and verified SMTP email.

5. User Guide for Testing

1. Start the UI with `python3 -m src.aussie_ecolens.server` and open `127.0.0.1:8080`.
2. Register or sign in through Cognito. After login, the app shows the user's name and protected tools.
3. In Image mode, upload one or more test images. Expected output: thumbnails, model-derived tag chips, duplicate feedback, and copyable Thumbnail URLs.
4. Query by tag count, for example species `casuarius_casuarius` with minimum count 1, then repeat as a species-only query.
5. Paste a Thumbnail URL into the thumbnail lookup tool. Expected output: an openable full original image URL.
6. Use Query by file to submit a file for temporary tag detection. The file is not stored as new media.

7. Add or remove manual tags from one or more media URLs, then query the tag to verify the update.
8. Switch to Video mode and upload a short video. Expected output: original video URL, extracted frame previews, and aggregated model tags.
9. Register a watch for a species tag, upload matching media, refresh Notifications, and confirm the SMTP email.
10. Delete a media URL and verify it disappears from query results and storage-derived outputs.

6. Team Contributions

Name and ID	Contribution %	Project elements contributed
TBD: Name, Student ID	TBD %	Replace with each member's real contribution summary, maximum 100 words per member.
TBD: Name, Student ID	TBD %	Replace with authentication, cloud deployment, ML integration, UI, testing, documentation, or reporting contributions as appropriate.
TBD: Name, Student ID	TBD %	Replace before final Moodle submission.

7. Repository and References

Private source code repository: https://github.com/ada-yq225/AussieEcoLens. Keep the repository private and share it with the teaching team.

Icon sources. AWS icons were downloaded from the official AWS Architecture Icons page: aws.amazon.com/architecture/icons. Google Cloud icons were downloaded from the official Google Cloud product icons library: cloud.google.com/icons.